Project 3: SSD Performance Profiling

ECSE 4320/6320 Advanced Computer Systems

Demetrios Sotirelis

October 10, 2026

Abstract

I profiled a Crucial MX500 1 TB SATA SSD (Windows 11, NTFS file on a dedicated 100 MiB test partition, direct I/O) with FIO 3.43 and an independent C++/Win32 reference tool. At queue depth 1 a 4 KiB random read takes 92 μs (p99 111 μs), a random write 36 μs (p99 148 μs), a 128 KiB sequential read 692 μs and write 281 μs. Throughput rises with queue depth until the drive's 32-slot NCQ is full: random reads peak at 99.5 k IOPS (408 MB/s) and random writes at 85 k IOPS, sequential reads at 564 MB/s and writes at 507 MB/s (the SATA III ceiling); beyond QD 32 IOPS stay flat and latency grows linearly, exactly as Little's Law $N = \text{IOPS} \times \text{latency}$ requires. The knee is predicted by the same law: $N^* = L_1 \times X_{\max}$ gives 8.8 (read), 3.4 (write), 2.5 and 1.2 outstanding requests, against observed 90% points at QD 16, 8, 4 and 2. A 70/30 read/write mix collapses to 40 k IOPS (41% of read-only and 48% of write-only), with p99 latency 1.9 ms. Buffered I/O, an unwritten file, or a synchronous engine that ignores the requested queue depth each inflate a "measurement" by 10–70×. Two things surprised me and are reported as anomalies rather than averaged away: the random-*read* ceiling is not constant but hops between plateaus (99.5 k, 95 k, 87 k and 65 k IOPS) and the drive stalls for ≈10 s at a time while sustained reads run, whereas writes are perfectly steady (Sec. 9). Temperature and SMART data need administrator rights and were unavailable, and the 80 MiB test file cannot exhaust the SLC cache or reveal capacity-driven steady-state behaviour; both limit what can be concluded about device state.

1 Summary of findings

Question	Answer (evidence)
QD=1 latency	4 KiB random read p50/p99 92/111 μs (10.8 k IOPS); random write 36/148 μs (26.2 k IOPS); 128 KiB sequential read 692/889 μs (186 MB/s); sequential write 281/395 μs (436 MB/s). The C++ reference tool agrees within 10% on every p50 (Tab. 1, Fig. 1).
Maximum IOPS / throughput	random 4 KiB read 99.5 k IOPS best plateau (408 MB/s), 85 k write; sequential read 564 MB/s, write 507 MB/s ≈ the SATA III payload ceiling; the datasheet figures (retailer listings of Crucial's data sheet) are 95 k/90 k IOPS and 560/510 MB/s (Tab. 2).
Block size	IOPS fall and MB/s rise with block size until the interface saturates: reads reach 558–564 MB/s from 64 KiB, writes 497–507 MB/s from 64–128 KiB; random ≈ sequential except at 4 KiB reads (anomaly).
Mix	4 KiB random, QD32: 100% read 99.3 k, 100% write 85.2 k, 70/30 40.5 k, 50/50 40.6 k IOPS (Tab. 3).
Queue depth / knee	90% of peak reached at QD16 (read, mean 177 μs), QD8 (write, 96 μs); peak at the NCQ depth (32); QD 64–256 add only latency (N = QD to 0.2%) (Tab. 4, Fig. 2).
Tail latency	read p99/p50 1.2–1.8 at QD 1–128 (p99.9 ≤ 2.4× p50); write p99.99 reaches 27–32 ms at QD≥32 because of ≈30 ms controller stalls (Tab. 5, Fig. 3).
Device state	150 s of sequential and 120 s of random writes at QD32 stay within 3–4.5% peak-to-peak of their first seconds; QD=1 latency before/after heavy writes and idle is identical; no SLC-cache cliff is reachable within 80 MiB (Fig. 4).
Misleading setups	buffered 4 KiB QD1 read 114.7 k vs. direct 10.9 k IOPS (10.6×); unwritten-file read 732 k vs. 10.9 k (67×); synchronous engine with `iodepth=32` 349 k (buffered, QD silently 1) (Sec. 7).
Anomaly	random-read ceiling hops 99.5 k → 65 k IOPS and the drive stalls ≈10 s every ≈190 s under sustained reads; writes are steady (Sec. 9).

2 AI-use note

Tools. Claude Code (Anthropic's coding agent) generated first drafts of the FIO job generator and orchestration script, the C++ reference tool, the parsing/plotting scripts, the OS-counter monitor and this report; it also installed FIO (extracted from the official MSI because installing it needs administrator rights). **Decisions that were mine:** using the dedicated `T:` partition, native Windows, no change to any system setting, and refining all three projects against the rubric. **What was caught or rejected after checking behaviour:**

- An over-strict free-space assertion in the safety check aborted the first run (before anything was written).
- FIO on Windows refuses direct I/O with the synchronous engines, so the planned "effective queue depth" pitfall jobs (`sync`, `iodepth=32`) were redesigned as buffered runs plus a `windowsaio` control (Sec. 7).
- The analysis script silently dropped groups with a missing `rwmixread`, mis-parsed the C++ tool's repeated CSV headers, and the first steady-state figure was unreadable; each was fixed and the numbers re-checked against raw JSON (a parser self-test on synthetic JSON, MB/s = IOPS × bs and Little's Law identities are in `sanity_check.py`).
- The first reading of "IOPS at QD32" was a single number; per-trial data showed that it varies by 15% between trials for reads, which turned into the anomaly study of Sec. 9 instead of an average.

I remain responsible for understanding and defending every claim.

3 Pre-experiment hypotheses

1. **QD=1 latency:** a SATA TLC drive reads a random 4 KiB page in roughly 80–120 μs; writes are acknowledged from the controller's buffer and should be faster (<50 μs); 128 KiB transfers add transfer time on top (128 KiB / 550 MB/s $\approx$ 240 μs), so I expected 128 KiB sequential read at QD1 to be $\approx$ 300–400 μs.
2. **Queue depth:** IOPS rise roughly linearly with QD until the device's internal parallelism or the NCQ depth (32 on SATA) is exhausted, then flatten while latency grows linearly (Little's Law); the knee should sit near $N^* = L_1 \times X_{\max}$.
3. **Block size:** IOPS fall and MB/s rise with block size until the 6 Gb/s SATA link ($\approx$550–560 MB/s payload) saturates; sequential $\geq$ random.
4. **Mix:** NAND reads wait behind program operations, so any mix with writes should be worse than a weighted average of the pure workloads.
5. **Tail:** writes have a worse tail than reads (garbage collection, flushing); the tail inflates sharply once the queue is saturated.
6. **Device state:** heavy sustained writes should eventually reduce write throughput (SLC cache exhaustion, GC); with only 80 MiB of LBA space I expected *no* cliff.
7. **Misleading setups:** buffered I/O and unwritten files should inflate IOPS by orders of magnitude.

Outcome: 2, 3 (except 4 KiB sequential reads), 4, 5, 6 and 7 confirmed in direction. Three surprises: 128 KiB sequential read at QD1 is 692 μs (186 MB/s), twice my estimate, because a QD1 direct read gets no read-ahead and pays the full flash access; the 70/30 mix is 41–48% of the *pure* workloads, not an average of them; and the read ceiling is not stable (Sec. 9).

4 Methodology and safe setup

Device	Crucial `CT1000MX500SSD1`, 1 TB, SATA III 6 Gb/s, firmware M3CR043, health "Healthy"; 3D TLC with a Silicon Motion SM2258 controller and dynamic SLC write acceleration (per public reviews; not verified here); NCQ depth 32; 512 B logical/physical sectors.
Target	**Dedicated 100 MiB NTFS partition `T:` (label "Testing")**, 1 MiB-aligned (offset mod 4096 = 0), cluster 4096 B, indexing/compression off; it lies at the end of the same physical disk as my data volume `G:`, which the benchmark *never opens*. One preallocated 80 MiB file `T:\ssdbench\fio.dat`; test range 64 MiB unless stated.
Data-loss safeguards	`run_fio.py` refuses to run unless `T:` is NTFS, labelled "Testing", ≤200 MiB, and contains nothing but `System Volume Information` and `ssdbench`; the C++ tool refuses any path outside `T:\`; no raw `\\.\PhysicalDrive`/volume handles, no formatting or partitioning.
I/O path	FIO 3.43, engine `windowsaio` (I/O completion ports), `direct=1` (`FILE_FLAG_NO_BUFFERING`), `thread`, `norandommap`, block sizes multiples of 4 KiB with block-aligned random offsets; every I/O refills its buffer with incompressible random data (`refill_buffers`, `buffer_compress_percentage=0`); the compressible/incompressible comparison is Tab. 6.
Preconditioning	the whole file is written twice sequentially before the first read test (NTFS returns zeros for never-written ranges without touching the SSD, Sec. 7); fresh-state experiments run first, heavy-write steady-state experiments last.
Host	AMD Ryzen 9 9950X (16C/32T, SMT on), 31 GiB RAM, Windows 11 build 26200, power plan High performance (recorded, not changed), FIO threads *not* pinned. Windows Defender real-time protection, Malwarebytes and NordVPN were running (their per-I/O filter cost is unmeasured); no other I/O on the disk before the runs (10×1 s check: 0 transfers/s).
Repetitions	3 trials per configuration (5 for QD1 baselines, 2 for 45 s tail runs), *trial-major* order (all configurations once, then again) with 2 s idle between jobs, so slow drift cannot bias one configuration; fixed per-trial `randseed`; tables show medians, plots the min–max across trials; each point runs 6–20 s after a 2 s ramp.
Not available	drive temperature, SMART and throttle status (need administrator rights): *thermal throttling can therefore not be observed directly*; OS counters (`PhysicalDisk`, per-CPU interrupt/DPC time) were used as a substitute for the anomaly study.

5 Correctness validation

1. **Independent reference tool.** `ref_qd1.cpp` (plain Win32 `ReadFile`/`WriteFile`, synchronous, one outstanding request, own timer and percentiles) reproduces the FIO QD1 numbers: 4 KiB random read p50 91.6 vs. 92 μs, write 33 vs. 36 μs(the largest difference, 7%), 128 KiB read 698 vs. 692 μs, write 274 vs. 281 μs(Tab. 1).
2. **Identities on all 331 runs:** MB/s = IOPS × block size holds to 1.07% (max deviation); N = IOPS × mean latency equals the requested iodepth with median ratio 0.998.
3. **Parser self-test:** `sanity_check.py` builds synthetic FIO JSON with known IOPS/percentiles and checks that the parser returns them (this caught the missing-percentile case).
4. **Physical sanity:** the 564 MB/s read plateau equals the SATA III payload limit (6 Gb/s × 8/10 = 600 MB/s raw, ≈560 after protocol overhead) and matches the published 560 MB/s; 99.5 k random-read IOPS is within 5% of the published 95 k.
5. **OS cross-check:** the `PhysicalDisk` counters report reads/s equal to FIO's IOPS (99 561 vs. 99 559; 87 065 vs. 87 080) and zero writes, so no hidden traffic competes (`isolation_check.csv`).

6 Results

6.1 Zero-queue baselines (QD=1)

QD=1 workload	IOPS	MB/s	mean	p50	p95	p99	p99.9	C++ ref p50 / p99 (μs)
			FIO latency (μs)					
4 KiB random read	10,842	44.4	92	92	94	111	136	92 / 105
4 KiB random write	26,220	107.4	38	36	38	148	163	33 / 149
128 KiB seq read	1,417	185.7	705	692	831	889	979	698 / 894
128 KiB seq write	3,329	436.3	298	281	379	395	514	274 / 397

Table 1: QD=1, direct I/O, 5 trials × 20 s, medians. Latency from FIO's completion-to-submission latency (`lat`); last column from the independent C++ tool (p50 / p99).

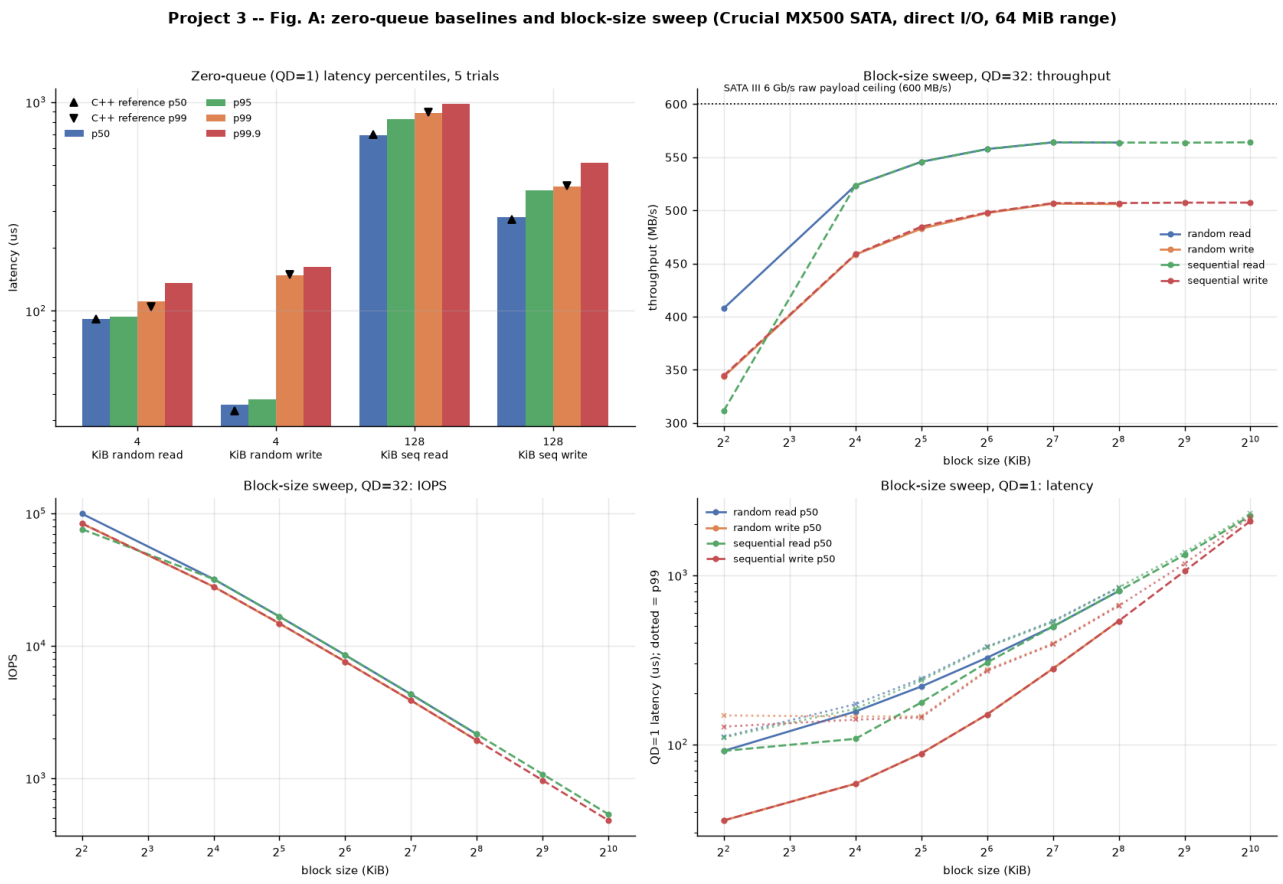


Figure 1: Top left: QD=1 latency percentiles (bars: FIO; triangles: C++ reference). Top right/bottom left: block-size sweep at QD=32 (throughput, IOPS). Bottom right: block-size sweep at QD=1 (p50 solid, p99 dotted). Fixed: direct I/O, 64 MiB range; varied: block size, pattern, direction.

A random 4 KiB read costs 92 μs with a very tight distribution (p99 only 20% above p50, p99.9 136 μs), so one core issuing synchronous reads is limited to 10.8 k IOPS (44 MB/s). Writes are 2.5× faster (36 μs) because the controller acknowledges data from its buffer before programming NAND. Their p99 is 4× the median (148 μs) while p95 is only 38 μs: between 1% and 5% of writes wait for something internal. Large sequential requests are transfer-dominated: 128 KiB takes 692 μs to read (186 MB/s, only a third of the link rate) but 281 μs to write (436 MB/s), i.e. a QD1 sequential *write* is 2.3× faster than a read of the same size, again the write buffer.

6.2 Block-size and access-pattern sweep

		QD=1			QD=32		
pattern	bs	IOPS	MB/s	p50 μs	IOPS	MB/s	mean μs
random read	4k	10,849	44	92	99,572	408	319
	16k	6,478	106	157	31,945	523	1001
	32k	4,647	152	220	16,647	546	1922
	64k	3,056	200	326	8,504	558	3761
	128k	2,004	263	498	4,298	564	7440
	256k	1,235	324	807	2,145	564	14895
random write	4k	26,272	108	36	83,865	344	380
	16k	15,329	251	59	27,971	458	1149
	32k	10,301	338	89	14,726	483	2172
	64k	5,899	387	151	7,585	497	4216
	128k	3,321	435	281	3,858	506	8326
	256k	1,754	460	537	1,925	506	16595
sequential read	4k	10,878	45	92	76,033	311	419
	16k	8,337	137	108	31,946	523	1001
	32k	5,326	175	177	16,647	546	1922
	64k	3,191	209	305	8,502	558	3762
	128k	2,013	264	498	4,297	564	7440
	256k	1,233	323	807	2,145	564	14895
	512k	757	397	1319	1,070	564	29826
	1m	443	464	2245	533	564	59753
sequential write	4k	26,466	108	36	84,124	345	378
	16k	15,442	253	59	27,998	459	1142
	32k	10,323	338	89	14,784	485	2163
	64k	5,890	386	151	7,594	498	4210
	128k	3,322	435	281	3,862	507	8278
	256k	1,751	459	537	1,928	507	16566
	512k	906	475	1057	962	507	33142
	1m	464	486	2089	479	507	66443

Table 2: Block sizes 4 KiB–256 KiB for random and sequential access, plus 512 KiB and 1 MiB for sequential only; medians of 3 trials. Throughput and latency come from the same runs at QD=1 and QD=32.

Throughput and IOPS tell different stories (Fig. 1): at QD32, 4 KiB random reads deliver 99.6 k IOPS but only 408 MB/s, while 64 KiB reads deliver 8.5 k IOPS and 558 MB/s: IOPS are limited by per-command overhead below ≈16 KiB and by the SATA link above it. The link ceiling (564 MB/s read) is reached at 64 KiB and the mean latency then grows in proportion to the block size (3.8 ms at 64 KiB, 14.9 ms at 256 KiB with 32 requests queued: transfer time × 32). Writes saturate lower (497–507 MB/s, the drive's sustained write rate). At QD1 the latency of every size is the sum of a fixed ≈90 μs (read) or ≈35 μs (write) command cost and size/rate, so MB/s grows with block size until the single outstanding request can no longer hide the access time (464 MB/s at 1 MiB). At QD32 random and sequential agree to a few percent for blocks ≥16 KiB, because the drive has no seek cost; the one exception is 4 KiB *sequential* read: 76 k vs. 99.6 k IOPS for random (Sec. 9). At QD1 sequential reads of 16–64 KiB are faster than random ones (p50 108 vs. 157 μs at 16 KiB, 305 vs. 326 μs at 64 KiB), consistent with the controller prefetching the next pages of a sequential stream (hypothesis, untested); writes are identical for both patterns.

6.3 Read/write mix

mix (4 KiB random)	QD	IOPS	MB/s	read p50 / p99 μs	write p50 / p99 μs
100R	1	10,633	44	92 / 123	–
100R	32	99,337	407	317 / 469	–
100W	1	25,704	105	–	36 / 148
100W	32	85,155	349	–	371 / 403
70R30W	1	12,829	53	91 / 127	36 / 76
70R30W	32	40,497	166	774 / 1925	692 / 1827
50R50W	1	14,729	60	90 / 142	36 / 79
50R50W	32	40,590	166	758 / 1942	676 / 1843

Table 3: 4 KiB random, fixed range; IOPS and MB/s are totals (read+write); latencies per direction.

At QD32 a pure read workload delivers 99.3 k and a pure write workload 85.2 k IOPS, yet 70% reads / 30% writes delivers only 40.5 k, and 50/50 40.6 k: the mix is 41–48% of either pure workload, and read p99 grows from 0.47 ms to 1.9 ms. A NAND die that is busy programming (hundreds of microseconds) cannot serve reads, so read and write streams block one another, and the controller's write-buffer flushes compete with reads. At QD1 the interference is mild (read p50 stays 90 μs), which shows that the damage is a queueing effect among concurrent requests. The same holds for 128 KiB sequential: read-only 4.3 k IOPS (564 MB/s) falls to 2.3 k reads + 1.0 k writes at 70/30 (426 MB/s total) and 1.5 k + 1.5 k at 50/50 (400 MB/s).

6.4 Queue-depth sweep, knee, and Little's Law

workload	QD1 IOPS	max IOPS	MB/s at max	$N^* = L_1 X_{\max}$	QD for 90%	mean latency there	QD for 98%	latency × at top QD
random 4K read	10,704	94,524	387	8.8	16	177 μs	32	32× (QD 256)
random 4K write	25,912	87,707	359	3.4	8	96 μs	16	79× (QD 256)
random 4K 70/30 mix	12,888	40,549	166	3.1	8	197 μs	8	82× (QD 256)
seq 128K read	1,395	3,521	462	2.5	4	1136 μs	4	25× (QD 64)
seq 128K write	3,308	3,876	509	1.2	2	535 μs	4	55× (QD 64)

Table 4: Knee analysis. $N^* = L_1 \times X_{\max}$: mean QD1 latency times maximum IOPS = outstanding requests needed to saturate (Little's Law). "QD for 90%/98%": first swept QD reaching that fraction of the maximum.

Shape. Random read IOPS grow almost linearly up to QD8 (10.7 k → 58.8 k at QD8, 5.5× for 8× the queue), bend at QD16 (89.1 k, 90% of the best plateau at a mean latency of only 177 μs, 1.9× idle) and stop at QD32 (the SATA NCQ depth). From there every additional request only waits: mean latency doubles with each doubling of QD (337 μs at 32, 732 μs at 64, 1.49 ms at 128, 2.94 ms at 256, ×32 vs. QD1) for no gain in IOPS (error bars show the plateau hopping of Sec. 9, not a QD effect). The numjobs sweep gives the same curve as iodepth, so "queue depth" means outstanding requests however they are generated. Writes saturate sooner (QD8 reaches 95% of the 85 k plateau, p99 only 212 μs) because the controller acknowledges them early; sequential 128 KiB saturates at QD4 (reads: 3.5 k IOPS = 462 MB/s in this stage) and QD2–4 (writes). The 70/30 mix saturates at QD8 at 40 k IOPS and has by far the worst tail (p99 1.1 ms already at QD8).

Little's Law. With the measured mean latency, $N = \text{IOPS} \times \text{latency}$ equals the requested iodepth to within 0.2% at every QD from 1 to 256 (bottom left of Fig. 2), which only confirms that fio keeps exactly iodepth requests in flight. The informative use is predictive: with L_1 the zero-queue latency and $X_{\max}$ the saturated rate, $N^* = L_1 X_{\max}$ is the number of requests the device must overlap to deliver $X_{\max}$ – the *internal parallelism* of the drive as seen from the host. It is 8.8 for random reads (93 μs × 94.5 k), 3.4 for writes, 2.5 for 128 KiB reads and 1.2 for 128 KiB writes; the observed knees (QD 16, 8, 4, 2) lie one doubling above because a short queue also needs slack to keep every flash channel busy while completions are processed. Beyond N^* extra requests only wait, so the mean latency is $\text{QD}/X_{\max}$ with $X_{\max}$ the plateau in force (QD256 read: 256/87.1 k = 2.94 ms, measured 2.94 ms). The practical operating point is therefore QD≈16 for reads and 8 for writes: 90% of the throughput at ≤6% of the QD256 latency (177 μs vs. 2.94 ms; 96 μs vs. 3.0 ms).

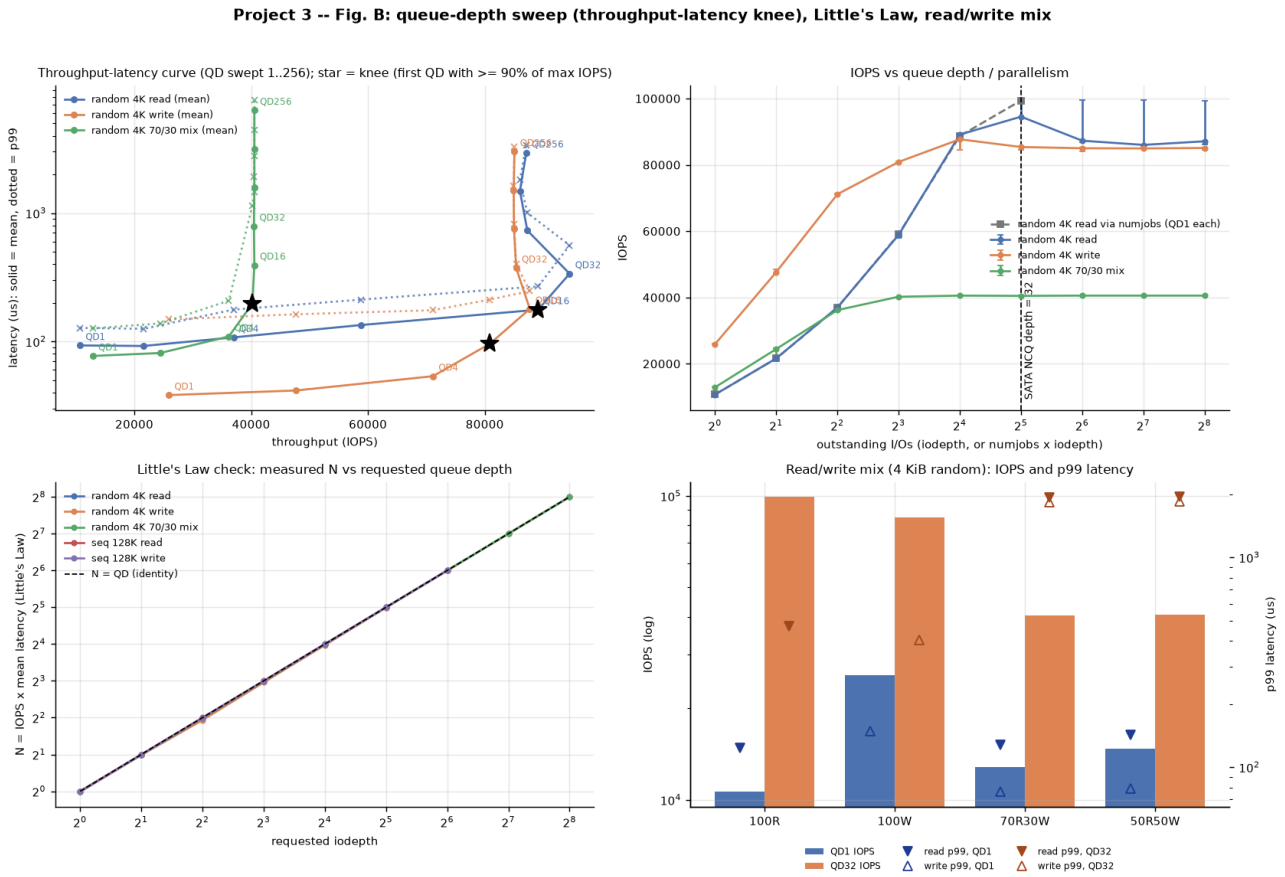


Figure 2: Top left: throughput–latency curve (mean solid, p99 dotted) for QD 1–256; stars mark the knee (first QD with $\geq$90% of maximum). Top right: IOPS vs. outstanding I/Os (error bars: min–max of 3 trials; grey: the same read workload obtained with `numjobs` instead of `iodepth`); the dashed line is the NCQ depth. Bottom left: $N = \text{IOPS} \times \text{mean latency}$ against requested iodepth. Bottom right: IOPS and p99 by mix.

6.5 Tail latency

workload	QD	IOPS, median (min–max of 2 trials)	p50	p95	p99	p99.9	p99.99	max	p99/p50
			latency (μs)						
random 4K read	1	10.6 k (10.6–10.6)	92	115	131	184	229	2964	1.4
	4	36.6 k (36.6–36.7)	99	155	181	240	346	3051	1.8
	16	74.7 k (62.1–87.3)	218	289	332	408	523	3253	1.5
	32	86.2 k (86.2–86.3)	346	528	614	717	836	3433	1.8
	128	86.0 k (85.9–86.1)	1466	1712	1810	1950	4260	4622	1.2
random 4K write	1	25.6 k (25.6–25.7)	36	55	148	166	231	30278	4.2
	4	69.3 k (68.2–70.4)	42	158	178	664	880	30653	4.2
	16	85.1 k (84.3–85.8)	180	210	253	326	2273	31057	1.4
	32	84.3 k (83.8–84.8)	375	389	420	542	27525	45064	1.1
	128	84.1 k (83.3–85.0)	1499	1573	1638	2884	32113	32379	1.1

Table 5: 45 s runs, 2 trials pooled by median; random 4 KiB. QD 1 and 4 are low load, 16 is mid-load, 32 is at the knee/NCQ depth, 128 is past it (QD32 and QD128 ran entirely on the 86,k read plateau). Where the two trials sit on different read plateaus (QD16: 62 k and 87 k IOPS) the median is their average (Sec. 9).

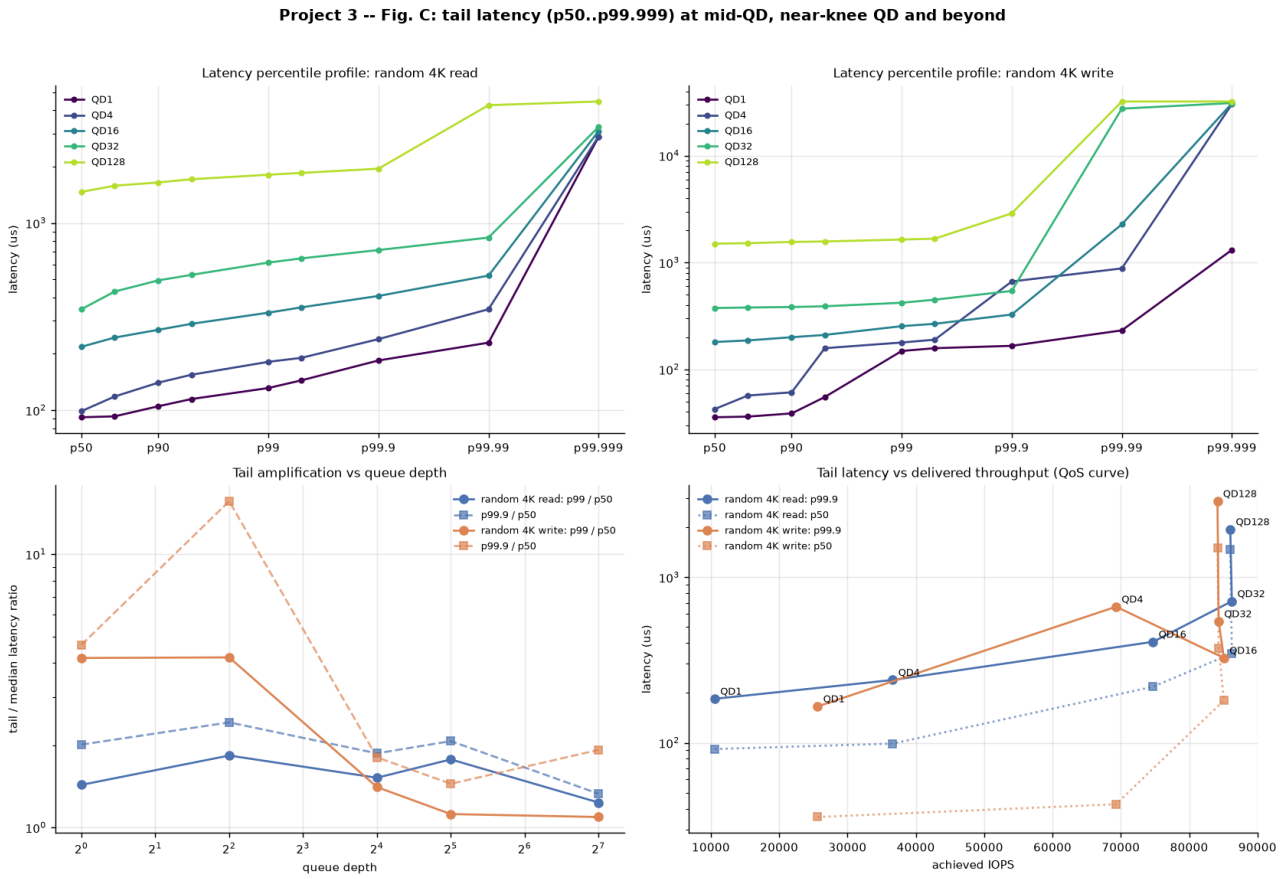


Figure 3: Percentile profiles from p50 to p99.999 (top), tail amplification (bottom left) and tail-vs-throughput trade-off (bottom right), random 4 KiB.

Reads have a compact tail: p99/p50 is 1.2–1.8 at every QD and p99.9 is at most 2.4× the median (QD32: p99.9 717 μs vs. p50 346 μs), because reads have a deterministic service time and queueing only shifts the whole distribution. Past the knee (QD128) the distribution is a shifted copy (p50 1.47 ms, p99 1.81 ms): the tail is queueing delay, ≈QD/IOPS. The rare outliers at p99.999 (≈3 ms at QD≤32) are independent of QD, i.e. not

queueing. **Writes** are different: tail amplification is largest at *low* load (p99/p50 4.2 at QD1 and QD4) because a minority of writes (p99 148 µs) hit an internal event; at QD≥16 the median grows (queueing) and the ratio shrinks to 1.1–1.4, but the extreme tail does not: p99.99 is 27–32 ms at QD32/128 and the maximum latency 30–45 ms at every QD, a ≈30 ms stall of the whole write path (also visible as the 30 ms maximum in the QD1 C++ reference: 28 ms). This is the signature of controller housekeeping (flushing the write buffer or garbage collection) and it is invisible to means and even to p99.9 at low QD. The QoS curve (bottom right) shows that queueing past the knee is pure loss: from QD32 to QD128 the p99.9 read latency grows 2.7× (717 µs→ 1.95 ms) and the write p99.9 5.3× for the same IOPS.

6.6 Working-set (LBA range) and device state

workload (QD=32, write)	payload	IOPS	MB/s	mean µs	p99 µs
4 KiB random	incompressible	83,761	343	380	395
	90% compressible	83,794	343	380	395
128 KiB sequential	incompressible	3,857	506	8289	8225
	90% compressible	3,851	505	8301	9241

Table 6: Data pattern: the payload's compressibility makes no measurable difference on this drive.

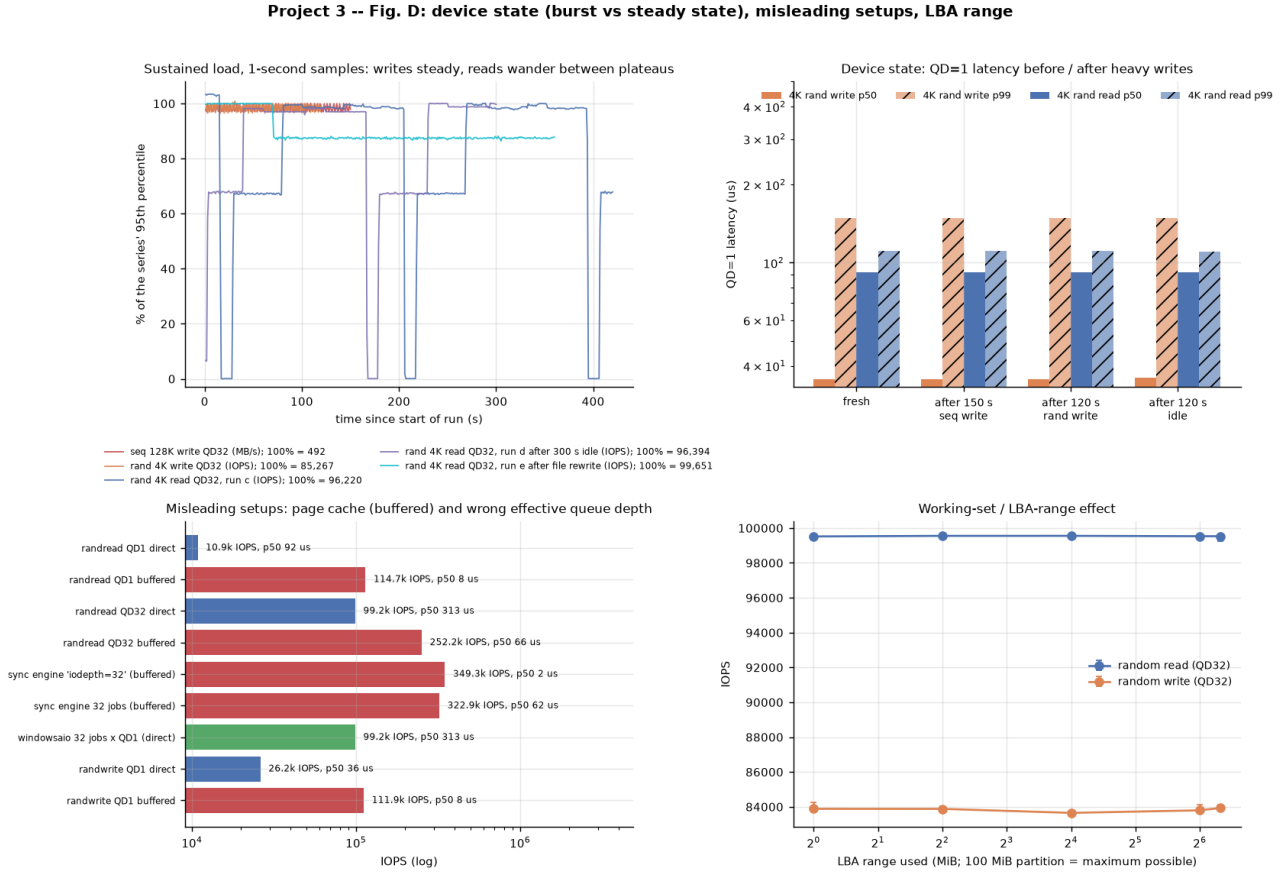


Figure 4: Top left: sustained load, 1-second samples, as a percentage of each series' 95th percentile. Top right: QD1 latency before and after heavy writes and a 120 s rest. Bottom left: misleading setups (blue: valid, red: misleading). Bottom right: random-access IOPS vs. LBA range.

LBA range. Random read IOPS (QD32) and write IOPS are identical from 1 MiB to 80 MiB (99.5 k and 83.8 k, differences <0.5%); the whole test file is far below the drive's DRAM-mapped and SLC-cached regions, so I

cannot show the capacity-dependent effects (mapping-table misses, SLC exhaustion) that appear on a full-drive span; this is the main limitation of using a 100 MiB partition and is the price of the data-loss safeguards. **Burst vs. steady state.** A 150 s sequential 128 KiB write at QD32 delivers 482 MB/s in the first 10 s and 482 MB/s in the last 30 s (minimum 477 MB/s); 120 s of random 4 KiB writes starts at 84.0 k and ends at 84.7 k IOPS (minimum 82.2 k). QD1 latency measured before and after the heavy writes and after a 120 s rest is identical (write p50 36 μs, read p50 92 μs, p99 111 μs). So *no burst-to-steady-state transition is observable* here; it is not evidence that the drive has none: 150 s $\times$ 482 MB/s = 72 GB written, but into the same 80 MiB of LBAs, which the controller can absorb in its SLC region. The 1-second write series oscillates (standard deviation 1.2% sequential, 1.4% random; peak-to-peak 3.2% and 4.5%; dominant period 3 s and 2 s), which is small next to the read behaviour discussed in Sec. 9. **Compressibility:** a 90%-compressible payload gives the same write IOPS and MB/s as incompressible data (Tab. 6); this drive shows no compression effect, so the data pattern is irrelevant here, but I kept the payload incompressible for all other experiments for comparability with drives that do compress.

7 Misleading setups (each measured)

1. **Page cache (buffered I/O).** The same 4 KiB random read with `direct=0`: 114.7 k IOPS at p50 7.9 μs vs. 10.9 k IOPS at 91.6 μs with direct I/O (10.6$\times$); at QD32 252 k vs. 99 k; random write at QD1 111.9 k vs. 26.2 k (4.3$\times$, writes complete into the cache). The C++ tool shows the same effect (310 k vs. 10.9 k reads, 328 k vs. 28 k writes). A buffered test measures RAM (8 μs is a memory copy plus system call), not the SSD; `FILE_FLAG_WRITE_THROUGH` alone does *not* bypass the read cache and on this drive gives the same 28.2 k IOPS as unbuffered writes (Tab. 7).
2. **Unwritten (sparse) file.** Reading a freshly extended NTFS file never touches the disk: 732 k IOPS at 1.3 μs vs. 10.9 k IOPS once the same range has been written (67$\times$). This is why the file is written twice before the first test. (NTFS valid-data-length semantics; measured with the C++ tool, `vdl` mode on a 6 MiB file.)
3. **Wrong effective queue depth.** FIO's synchronous engines ignore `iodepth`: `ioengine=sync` with `iodepth=32` gives 349 k IOPS (buffered, since FIO refuses direct I/O with synchronous engines on Windows), indistinguishable from QD1 buffered and 3.5$\times$ the real SSD's 99 k; 32 synchronous jobs reach 323 k, again cache. The correct way to obtain 32 outstanding I/Os is `windowsaio` with either `iodepth=32` (99.16 k IOPS) or 32 jobs of QD1 (99.16 k): identical, as Little's Law predicts.
4. **Single-trial or averaged read results.** The median of three QD32 read trials, 94.5 k IOPS, hides a spread of 94.1–99.3 k; at QD$\geq$64 the three trials span 86–99.5 k; the true distribution is multi-modal (Sec. 9), so one run, or one mean, would be an arbitrary pick among plateaus.

C++ reference (QD=1, synchronous)	ops/s	mean μs	p50	p99	p99.9
randread4k_buffered	310,294	3.2	2.1	10.9	98.1
randread4k_direct	10,945	91.3	91.6	104.9	159.4
randwrite4k_buffered	327,587	3.0	2.5	8.7	12.0
randwrite4k_direct	28,227	35.4	33.2	148.8	162.9
randwrite4k_wt	28,230	35.4	33.3	149.2	163.3
seqread128k_direct	1,417	705.8	697.6	893.6	994.3
seqwrite128k_direct	3,413	293.0	273.5	397.2	422.2
vdl_unwritten_file	732,188	1.3	1.3	1.5	2.0
vdl_written_file	10,929	91.5	91.4	103.0	169.9

Table 7: Independent C++ tool (3 runs $\times$ 8–10 s per mode, medians): buffered and unwritten-file cases are the misleading ones.

8 Reproducibility and data hygiene

Everything needed to rebuild the results is in the repository: `scripts/run_fio.py` (all stages, resume-safe), `scripts/run_all.ps1`, FIO job files for every run (`data/raw/jobs`), raw FIO JSON (`data/raw/json`) and per-second logs (`data/raw/logs`), the C++ tool outputs (`data/raw/ref`), parsed CSVs (`data/processed`), the

plotting script and `results/environment.txt`. Median coefficient of variation of IOPS across trials is 0.1–1.0% in every stage (0.4% block-size, 0.3% mix and QD, 0.1% LBA range, 1.0% QD1 baselines). The largest values (6–24%) are reads at QD≥16 whose plateau hops (up to 24%, Sec. 9) and a few write configurations with one slow trial (128 KiB random write at QD32: 2.9 k vs. 3.86 k IOPS, 57 ms maximum; sequential writes of 512 KiB and of 128 KiB at QD1: 6–8%).

9 Honest anomalies

9.1 The random-read ceiling is not constant

Observation. 4 KiB random reads at QD≥32 do not have one throughput. In the main campaign the first trial of the QD sweep delivered 99.3–99.5 k IOPS at QD 32–256 while trials 2 and 3 delivered 86–94 k; the tail stage (QD 16/32/128) saw 62 k/86 k/86 k; stages run at other times (block size, mix, LBA range) saw 99.5 k. I therefore ran dedicated 1-second-resolution reads (Fig. 4, top left; `time_series_modes.csv`):

- *Run c (420 s):* 99.4 k for 10 s, then a **stall** (reads/s drop to 60–90, mean read latency 0.37 s with 32 requests outstanding) for ≈12 s, then a plateau at 64.7 k for 45–50 s, then 94.7–95.7 k for ≈115 s, then a stall again at t ≈207 s and t ≈394 s: a cycle of ≈190 s. 36% of the seconds were below 75% of the 95th percentile and 8.6% below 25%.
- *Run d (300 s, after 300 s idle):* 51 k, 65 k (30 s), 94 k (110 s), one stall at t ≈168 s, 65 k (50 s), 95–96 k.
- *Run a (240 s, started a minute after another read run):* a constant 87 k for the whole run. *Run e (after rewriting the whole file twice):* 99.6 k for 66 s, one step to 87.1 k, then constant for 290 s with no stall.
- The same behaviour appears for sequential reads (128 KiB QD≥4: 564 MB/s in the block-size stage, 462 MB/s in the QD stage). **Writes show none of it**: 150 s of sequential and 120 s of random writes stay within 3–4.5% peak-to-peak.

Host-side checks. The OS `PhysicalDisk` counters report reads/s equal to FIO's IOPS (to 1%), zero writes, and 30–32 requests outstanding at the device during every plateau; mean latency is QD/IOPS in each (300 µs at 94 k, 500 µs at 65 k, 370 ms during a stall); no other process performs substantial disk I/O (`s10_os_monitor_d.csv.procs.txt`). In run e (`os_monitor_summary.csv`) the whole machine used 3.3% CPU (about one of 32 logical CPUs), the busiest CPU 55% (maximum 77%), the interrupt vector of the SATA controller sat on CPU 1 at 20% (maximum 36%) and DPC time on the busiest DPC CPU peaked at 36% spread over five CPUs; the step from 99.6 k to 87 k at $t = 66$ s is invisible in every host counter. So the host is neither saturated nor changing state: *the throughput change originates in the drive or its link*. The observations are inconsistent with a simple one-parameter story, so I do not claim a cause:

- **Thermal or power throttling of the controller** would explain stepwise reductions and a limit cycle, and run b/c/d start fast after a rest; but run a started slow and stayed slow for 240 s, and run e shows a single step without recovery in 290 s. I could not read the temperature (administrator rights) to test it.
- **Firmware background work** (read-disturb management that relocates heavily read blocks, refresh, scrubbing) would explain stalls: 80 MiB read at 90 k IOPS hits the same flash pages ≈850 times per 190 s; relocations also change how the logical blocks are spread over channels and dies, and with it the achievable parallelism (the plateaus), which would explain different levels after different relocations.

Consequence for the report. I never quote a mean over plateaus as a capability: the maximum IOPS in Tab. 4 is the median of three trials (94.5 k at QD32, trials 94.1–99.3 k) and Fig. 2 shows the min–max; the headline "99.5 k" is the best plateau and "87 k" the long-run plateau; the tail-latency row for QD16 reads averages a 62 k and an 87 k trial (Tab. 5). Consequently any SSD QD sweep should be run with interleaved repeated trials and per-second logs.

9.2 Other anomalies

- **4 KiB sequential reads are slower than random reads at QD32** (76 k vs. 99.6 k IOPS), although every larger size is identical for both. Hypothesis (not tested): four consecutive 4 KiB LBAs share a 16 KiB flash page and die, so a sequential stream keeps fewer dies busy than random addresses; from 16 KiB up one request covers a whole page and the effect disappears.

- **Writes complete in 36 μs at QD1**, far below a NAND program time (hundreds of μs): the drive acknowledges from its volatile write cache. `FILE_FLAG_WRITE_THROUGH` gives the same 28.2 k IOPS as ordinary unbuffered writes (Tab. 7), so on this setup the 36 μs is a *cache acknowledgement*, not durable-write latency; a durable test (flush after every write) would be needed and was not run.
- **≈30 ms write stalls** (maximum latency 30–45 ms at every QD, p99.99 27–32 ms at QD≥32) and a small oscillation (period 2–3 s, 3–4.5% peak-to-peak) in the 1-second write series: periodic controller housekeeping (hypothesis: write-buffer flush); one trial of 128 KiB random writes at QD32 was 25% slower (2.9 k vs. 3.86 k IOPS, 57 ms maximum), which I attribute to the same mechanism but did not repeat.
- **A 128 KiB sequential read at QD1 reaches only 186 MB/s,** one third of the link rate and 2.3× slower than a write of the same size; with no read-ahead a direct QD1 read pays the flash access and the full transfer before the next one is issued.
- **No burst-to-steady-state transition** was observed, contrary to the textbook SSD story; this is a property of the 80 MiB range, not of the drive (Sec. 6.6).

10 Limitations

- **Capacity.** The 80 MiB file cannot show full-drive effects (SLC-cache exhaustion, garbage collection under capacity pressure, mapping-table misses); no "steady state" in the SSD-industry sense was reached. Choosing safety over a larger span was deliberate (the same physical drive holds my data).
- **Temperature/SMART** unavailable (no administrator rights): thermal throttling and write amplification could not be observed.
- **One drive, one host, one filesystem:** NTFS overhead and the Windows storage stack (minifilters of Defender/Malwarebytes/NordVPN) are included in every number, the effective queue depth is capped by the SATA NCQ, and none of the conclusions apply to NVMe drives with deep queues without further measurement. FIO threads were not pinned.
- **Optional extensions** (GPU/CUDA) were not attempted.